

# LLM Core Concepts – AI Mini Projects Suite

---

## Author

**Name:** Dharshini.A

## 1.Scaled Dot-Product Attention Implementation

### Overview

This project implements the **core mechanism of Transformer models called Attention Mechanism**. It explains how models like GPT understand relationships between words by assigning importance scores to each word in a sentence.

It is based on the research paper “**Attention Is All You Need**” (2017).

---

### Key Skills Covered

- **Attention Mechanism**
  - **Linear Algebra (Matrix Multiplication)**
  - **Query–Key–Value (QKV) Model**
  - **Softmax Function**
  - **Neural Network Foundations**
  - **Transformer Architecture Basics**
-

## Working Process

- Input sentence is converted into **word embeddings**
- **Query (Q), Key (K), Value (V)** matrices are generated
- Attention scores are calculated using  $Q \times K^T$
- Scores are scaled using  $\sqrt{dk}$  to stabilize gradients
- **Softmax** converts scores into probabilities
- Final weighted sum produces **context-aware output**

---

## Outcome

This project shows how AI focuses on important words in a sentence instead of treating all words equally.

## Output Screenshots

### Screenshot 1 – Input & Embeddings

```
PS C:\Users\ELCOT\OneDrive\Desktop\llm-core-concepts-python> python main.py
=====
INPUT SENTENCE
=====
['AI', 'transforms', 'the', 'world']

=====
WORD EMBEDDINGS
=====
[[0.62 0.11 0.45 0.72]
 [0.91 0.53 0.27 0.34]
 [0.15 0.87 0.64 0.29]
 [0.73 0.21 0.88 0.55]]

=====
QUERY MATRIX (Q)
=====
[[1.12 1.08 0.6 1.03]
 [0.87 1.21 0.76 1.33]
 [0.82 0.79 0.23 1.52]
 [1.29 1.47 0.66 1.57]]

=====
KEY MATRIX (K)
=====
[[0.89 0.73 0.83 0.58]
 [0.93 0.78 0.81 0.66]
 [1.04 0.72 0.62 0.74]
 [1.08 1.13 0.89 0.84]]

=====
VALUE MATRIX (V)
=====
[[0.32 1.3 0.91 1.33]
 [0.34 1.27 1.36 1.39]
 [0.43 0.74 0.85 1.24]
 [0.36 1.51 1.05 1.77]]

=====
RAW QK^T MATRIX
=====
```

## Screenshot 2 – Q, K, V Matrices

```
PS C:\Users\ELCOT\OneDrive\Desktop\llm-core-concepts-python> python main.py
RAW QK^T MATRIX
=====
[[2.87 3.04 3.08 3.82]
 [3.05 3.24 3.23 4.09]
 [2.36 2.55 2.69 3.25]
 [3.67 3.91 3.98 4.95]]

=====
SCALED SCORES
=====
[[1.43 1.52 1.54 1.91]
 [1.52 1.62 1.62 2.05]
 [1.18 1.28 1.34 1.62]
 [1.83 1.95 1.99 2.48]]

=====
SOFTMAX PROBABILITIES
=====
[[0.21 0.23 0.23 0.33]
 [0.2  0.23 0.22 0.35]
 [0.21 0.23 0.24 0.32]
 [0.19 0.22 0.22 0.37]]

=====
FINAL ATTENTION OUTPUT VECTOR
=====
[[0.36 1.24 1.05 1.47]
 [0.36 1.24 1.05 1.48]
 [0.36 1.23 1.04 1.46]
 [0.36 1.25 1.05 1.48]]

=====
EXPLANATION
=====
1. Word embeddings represent each word numerically.
2. Q, K, and V matrices are generated using weight matrices.
3.  $QK^T$  calculates similarity between words.
4. Scores are scaled using  $\sqrt{d_k}$ .
5. Softmax converts scores into probabilities.
6. Final attention output is computed using probabilities and V matrix.
```

## 2.Tokenization and Context Window Cost Analyzer

### Overview

This project demonstrates how **text is broken into tokens** before being processed by Large Language Models. It also shows how token usage impacts **cost and context limits** in AI systems.

---

### Key Skills Covered

- **Tokenization (BERT Tokenizer)**
- **Natural Language Processing (NLP)**
- **Context Window Management**
- **Cost Estimation in LLM APIs**
- **Text Preprocessing**
- **Subword Tokenization**

---

### Working Process

- Input text is converted into **tokens using Hugging Face BERT tokenizer**
  - Total number of tokens is calculated
  - Token length is analyzed for efficiency
  - API cost is estimated based on token usage
  - Context window overflow is detected
-

## □ Outcome

This project explains why long prompts are expensive and how LLMs have memory limits.

## Output Screenshots

```
HALLUCINATION DETECTION AND PROMPT FIX
=====
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 148/148 [00:00<00:00, 1520.39it/s]

BEFORE PROMPT FIX
=====
[transformers] Passing `generation_config` together with generation-related arguments=(('max_new_tokens', 'temperature', 'do_sample')) is deprecated and will be removed in future versions. Please pass either a `generation_config` object OR all generation parameters explicitly, but not both.
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=80) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main\_classes/text\_generation)
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force cleanup anyway.
List Supreme Court AI judgments in India 2025. The AI decision states that AI will be used in courts with high judicial functioning and will not rely solely on the Supreme Court's deliberations.

Read more: AI to create AI judges for Delhi, Bihar, Jharkhand

This week, the state government announced that it plans to extend the Indian Computer Science degree program from three years to 2026. The new degrees would bring AI up

AFTER PROMPT FIX
=====
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=80) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main\_classes/text\_generation)

You are a factual AI assistant.

Rules:
1. Do NOT invent legal cases or citations.
2. If verified information is unavailable, clearly say so.
3. Only provide confirmed information.
4. Return output strictly in JSON format.

Question:
List verified Supreme Court AI judgments in India in 2025.

Answer:
```

## Screenshot 2 – Token Count & Cost

Answer:

1. The Supreme Court has been in existence since 1891.
2. The Supreme Court is the only Supreme Court in India to have a single Supreme Court.
3. The Supreme Court has been in existence since 1891.
4. The Supreme Court has been in existence since 1891.
5. The Supreme Court has been in existence

=====

OBSERVATION

=====

Before Prompt Fix:

- The model may generate hallucinated legal acts, fake citations, or unverified judgments.
- High randomness increases misinformation risk.

After Prompt Fix:

- Structured prompting improves reliability.
- Low temperature reduces randomness.
- JSON constraints encourage organized output.
- The model is more likely to admit uncertainty instead of inventing fake legal information.

=====

FINAL RECOMMENDATION

=====

For legal, medical, and financial systems:

- Use strict prompts
- Use low temperature
- Ask for structured JSON output
- Instruct the model to avoid assumptions
- Require verified information only

### Screenshot 3 – Context Window Overflow

```
=====
OBSERVATION
=====

Before Prompt Fix:
- The model may generate hallucinated legal acts,
  fake citations, or unverified judgments.
- High randomness increases misinformation risk.

After Prompt Fix:
- Structured prompting improves reliability.
- Low temperature reduces randomness.
- JSON constraints encourage organized output.
- The model is more likely to admit uncertainty
  instead of inventing fake legal information.

=====
FINAL RECOMMENDATION
=====

For legal, medical, and financial systems:

- Use strict prompts
- Use low temperature
- Ask for structured JSON output
- Instruct the model to avoid assumptions
- Require verified information only

This helps reduce hallucinations and improves
trustworthiness in critical AI applications.

=====
PROGRAM COMPLETED SUCCESSFULLY
=====
```

# 3.Sampling Strategy Experiment

## Overview

This project demonstrates how AI generates different responses using various **decoding strategies**. It explains how randomness affects creativity in AI-generated text.

---

## Key Skills Covered

- Text Generation (GPT-2 Model)
  - Temperature Sampling
  - Top-k Sampling
  - Top-p (Nucleus) Sampling
  - Probability Distribution
  - AI Creativity Control
- 

## Working Process

- GPT-2 model generates text output
  - Different sampling strategies are applied:
    - Low temperature → more deterministic output
    - High temperature → more creative output
    - Top-k → limits vocabulary choices
    - Top-p → dynamic probability filtering
  - Outputs are compared side-by-side
-

## Outcome

This project shows how AI balances between **accuracy** and **creativity** during text generation.

## Output Screenshots

```
SAMPLING STRATEGY EXPERIMENT
=====
Warning: you are sending unauthenticated requests to the HF Hub. Please set a HF TOKEN to enable higher rate limits and faster downloads.
config.json: 100%|██████████████████████████████████████████████████████████████████████████████| 665/665 [00:00<00:00, 926kB/s]
C:\Users\ELCOT\AppData\Local\Programs\Python\Python314\Lib\site-packages\huggingface_hub\file_download.py:138: UserWarning: 'huggingface_hub' cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\ELCOT\.cache\huggingface\hub\models--gpt2. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING' environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations. To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
  warnings.warn(message)
model.safetensors: 100%|██████████████████████████████████████████████████████████████████████████████| 549M/549M [02:06<00:00, 4.32MB/s]
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 148/148 [00:00<00:00, 3326.57it/s]
generation_config.json: 100%|██████████████████████████████████████████████████████████████████████████████| 124/124 [00:00<00:00, 347kB/s]
tokenizer_config.json: 100%|██████████████████████████████████████████████████████████████████████████████| 26.0/26.0 [00:00<00:00, 76.96kB/s]
vocab.json: 100%|██████████████████████████████████████████████████████████████████████████████| 1.04M/1.04M [00:00<00:00, 4.40MB/s]
merges.txt: 100%|██████████████████████████████████████████████████████████████████████████████| 456k/456k [00:00<00:00, 9.34MB/s]
tokenizer.json: 100%|██████████████████████████████████████████████████████████████████████████████| 1.39V/1.39M [00:00<00:00, 7.87MB/s]

PROMPT:
Explain Artificial Intelligence in simple terms.

=====
TEMPERATURE = 0.1
=====
[transformers] Passing 'generation_config' together with generation-related arguments('{{do_sample', 'max_new_tokens', 'temperature}}') is deprecated and will be removed in future versions. Please pass either a 'generation_config' object OR all generation parameters explicitly, but not both.
[transformers] Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
[transformers] Both 'max_new_tokens' (>=0) and 'max_length' (<=0) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Ignoring clean up tokenization spaces=True for BPE tokenizer GPT2Tokenizer. The clean up tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even though it will corrupt output=True to force cleanup anyway.
Explain Artificial Intelligence in simple terms.

The idea is to create a system that can be used to solve problems in a way that is easy to understand and understand.

The system is called a "machine learning system."
```

## Screenshot 2 – Temperature 1.0 Output

```
=====
TEMPERATURE = 1.0
=====
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=50) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Explain Artificial Intelligence in simple terms.

"The most obvious of all these things is the ability to detect whether a machine is aware of the human side of how we make decisions," he wrote. "The most fundamental problem of all – which is our ability to reason and interpret our

=====
TOP-K = 10
=====
[transformers] Passing `generation_config` together with generation-related arguments=({'top_k', 'do_sample', 'max_new_tokens'}) is deprecated and will be removed in future versions. Please pass either a `generation_config` object OR all generation parameters explicitly, but not both.
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=50) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Explain Artificial Intelligence in simple terms.

A simple example of this idea could be a "computer" that learns to read human speech. The computer would be able to recognize human words, or understand human language, to determine which of its own words are correct. It would then use

=====
TOP-P = 0.9
=====
[transformers] Passing `generation_config` together with generation-related arguments=({'top_p', 'max_new_tokens', 'do_sample'}) is deprecated and will be removed in future versions. Please pass either a `generation_config` object OR all generation parameters explicitly, but not both.
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=50) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Explain Artificial Intelligence in simple terms. If you're a programmer, you have to learn how to use algorithms and make sure that you understand what you're doing and what's going on.

I know there's a lot of people out there who are not familiar with AI, but

=====
COMPARISON SUMMARY
=====
```

### Screenshot 3 – Top-k Output

```
=====
COMPARISON SUMMARY
=====

Temperature = 0.1
- Produces stable and predictable responses
- Less creativity
- More accurate and focused

Temperature = 1.0
- Produces more creative and diverse responses
- Higher randomness
- Better for storytelling and brainstorming

Top-k Sampling
- Limits token selection to top probable words
- Produces controlled creativity
- Helps reduce strange outputs

Top-p Sampling
- Dynamically selects probable tokens
- Produces more natural language generation
- Balances creativity and coherence

=====
FINAL RECOMMENDATION
=====

Recommended Usage:

1. Legal Systems
- Use Low Temperature (0.1)
- Use Smaller Top-k
Reason:
Legal systems require accurate, deterministic,
and consistent outputs with minimal randomness.

2. Creative Writing Systems
- Use Higher Temperature (1.0)
- Use Top-p Sampling
```

#### Screenshot 4 – Top-p Output

```
=====
FINAL RECOMMENDATION
=====

Recommended Usage:

1. Legal Systems
- Use Low Temperature (0.1)
- Use Smaller Top-k
Reason:
Legal systems require accurate, deterministic,
and consistent outputs with minimal randomness.

2. Creative Writing Systems
- Use Higher Temperature (1.0)
- Use Top-p Sampling
Reason:
Creative applications benefit from diverse,
imaginative, and flexible text generation.

=====
PROGRAM COMPLETED SUCCESSFULLY
=====
```

## 4.Hallucination Detection and Prompt Fix

### Overview

This project demonstrates how AI can sometimes generate **incorrect or fabricated information (hallucination)** and how structured prompts improve output accuracy.

---

### Key Skills Covered

- **Prompt Engineering**
  - **AI Hallucination Understanding**
  - **Output Validation Techniques**
  - **Instruction Design**
  - **LLM Behavior Control**
  - **Structured Prompting (JSON-style prompts)**
- 

### Working Process

- A vague prompt is given to the model
  - AI generates incorrect or uncertain output
  - Prompt is improved using constraints:
    - clear instructions
    - structured format
    - limited scope
  - Output is regenerated and compared
-

## **Outcome**

This project shows how better prompting improves AI reliability and reduces hallucinations.

### Screenshot 1 – Before Prompt Fix

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF\_TOKEN to enable higher rate limits and faster downloads.  
Loading weights: 100% | 148/148 [00:00<00:00, 1520.39it/s]

```
[transformers] Passing `generation_config` together with generation-related arguments=({'max_new_tokens', 'temperature', 'do_sample'}) is deprecated and will be removed in future versions. Please pass either a `generation_config` object OR all generation parameters explicitly, but not both.
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=80) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main\_classes/text\_generation)
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force clean up anyway.
List Supreme Court AI judgments in India 2025. The AI decision states that AI will be used in courts with high judicial functioning and will not rely solely on the Supreme Court's deliberations.
```

This week, the state government announced that it plans to extend the Indian Computer Science degree program from three years to 2026. The new degrees would bring AI up

```
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=80) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
```

### Rules:

1. Do NOT invent legal cases or citations.
2. If verified information is unavailable, clearly say so.
3. Only provide confirmed information.
4. Return output strictly in JSON format.

List verified Supreme Court AI judgments in India in 2025.

Answer:

## Screenshot 2 – Hallucinated Output

Answer:

1. The Supreme Court has been in existence since 1891.
2. The Supreme Court is the only Supreme Court in India to have a single Supreme Court.
3. The Supreme Court has been in existence since 1891.
4. The Supreme Court has been in existence since 1891.
5. The Supreme Court has been in existence

=====

OBSERVATION

=====

Before Prompt Fix:

- The model may generate hallucinated legal acts, fake citations, or unverified judgments.
- High randomness increases misinformation risk.

After Prompt Fix:

- Structured prompting improves reliability.
- Low temperature reduces randomness.
- JSON constraints encourage organized output.
- The model is more likely to admit uncertainty instead of inventing fake legal information.

=====

FINAL RECOMMENDATION

=====

For legal, medical, and financial systems:

- Use strict prompts
- Use low temperature
- Ask for structured JSON output
- Instruct the model to avoid assumptions
- Require verified information only

### Screenshot 3 – After Prompt Fix

```
=====
OBSERVATION
=====
```

Before Prompt Fix:

- The model may generate hallucinated legal acts, fake citations, or unverified judgments.
- High randomness increases misinformation risk.

After Prompt Fix:

- Structured prompting improves reliability.
- Low temperature reduces randomness.
- JSON constraints encourage organized output.
- The model is more likely to admit uncertainty instead of inventing fake legal information.

```
=====
FINAL RECOMMENDATION
=====
```

For legal, medical, and financial systems:

- Use strict prompts
- Use low temperature
- Ask for structured JSON output
- Instruct the model to avoid assumptions
- Require verified information only

This helps reduce hallucinations and improves trustworthiness in critical AI applications.

```
=====
PROGRAM COMPLETED SUCCESSFULLY
=====
```

## 5.Semantic Search Engine Using Embeddings

### Overview

This project demonstrates how modern AI understands **meaning (semantics)** instead of just matching keywords.

It uses **vector embeddings** to find the most relevant answer based on similarity.

---

### Key Skills Covered

- Sentence Embeddings (MiniLM Model)
  - Cosine Similarity
  - Vector Search
  - Semantic Search Systems
  - Information Retrieval (IR)
  - AI Recommendation Systems
- 

### Working Process

- FAQs are converted into **embedding vectors**
  - User query is also converted into embeddings
  - **Cosine similarity** is calculated between vectors
  - Highest similarity score is selected
  - Most relevant answer is returned
-

## Outcome

This project shows how AI understands **meaning instead of exact words**, similar to Google Search and ChatGPT retrieval systems.

## Output Screenshots

### Screenshot 1 – Similarity Scores

## SEMANTIC SEARCH ENGINE USING EMBEDDINGS

USER QUERY:

How can I reset my password?

Loading embedding model...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF\_TOKEN to enable higher rate limits and faster downloads.

```
modules.json: 100% | 349/349 [00:00<00:00, 1.02MB/s]
```

C:\Users\ELCOT\AppData\Local\Programs\Python\Python314\Lib\site-packages\huggingface\_hub\file\_download.py:138: UserWarning: `huggingface\_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\ELCOT\.cache\huggingface\hub\models--sentence-transformers--all-MiniLM-L6-v2. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF\_HUB\_DISABLE\_SYMLINKS\_WARNING` environment variable. For more details, see [https://huggingface.co/docs/huggingface\\_hub/how-to-cache#limitations](https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations).

To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: <https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development>

```
warnings.warn(message)
```

```
config_sentence_transformers.json: 100%| 116/116 [00:00<00:00, 370kB/s]
```

|                 |                                     |
|-----------------|-------------------------------------|
| README.md: 100% | 10.5k/10.5k [00:00<00:00, 27.6MB/s] |
|-----------------|-------------------------------------|

```

sentence bert config.json: 100% | 53.0/53.0 [00:00<00:00, 127kB/s]

```

```
config.json: 100% | 612/612 [00:00<00:00, 2.74MB/s]
```

```
model.safetensors: 100% | 90.9M/90.9M [00:40<00:00, 2.24MB/s]
```

```

Loading weights: 100% | 103/103 [00:00<00:00, 2849.58it/s]

```

```
tokenizer_config.json: 100% | 350/350 [00:00<00:00, 984kB/s]
```

vocab.txt: 100% | 232k/232k [00:00<00:00, 1.17MB/s]

```
tokenizer.json: 100% |██████████| 466k/466k [00:00<00:00, 5.53MB/s]
```

```
special tokens map.json: 100% | 112/112 [00:00<00:00, 427kB/s]
```

```
config.json: 100% | 190/190 [00:00<00:00, 398kB/s]
```

## SIMILARITY SCORES

FAO 1: 1.0000

Question: How can I reset my password?

FAQ 2: 0.4091

Question: How do I create a new account?

FAQ 3: 0.4187

Question: How can I update my email address?

## Screenshot 2 – Top Match Result

-----  
FAQ 4: 0.3755

Question: How do I contact customer support?  
-----

FAQ 5: 0.3304

Question: How can I change my profile picture?  
-----

FAQ 6: 0.5742

Question: How do I delete my account?  
-----

FAQ 7: 0.1574

Question: How can I track my order?  
-----

FAQ 8: 0.3893

Question: How do I cancel my subscription?  
-----

FAQ 9: 0.2975

Question: How can I enable two-factor authentication?  
-----

FAQ 10: 0.1485

Question: How do I download my invoice?  
-----

=====

TOP MATCH

=====

Best Matching Question:

How can I reset my password?

Similarity Score:

1.0000

Best Answer:

To reset your password, click on Forgot Password and follow the instructions.

=====

KEYWORD SEARCH RESULT

=====

Exact Keyword Match Found:

How can I reset my password?

### Screenshot 3 – Keyword vs Semantic Comparison

```
=====
KEYWORD SEARCH RESULT
=====
Exact Keyword Match Found:
How can I reset my password?
To reset your password, click on Forgot Password and follow the instructions.

=====
COMPARISON
=====

Semantic Search:
- Understands meaning and context
- Uses embeddings and cosine similarity
- Can match similar sentences even with different wording
- Better for intelligent AI systems

Keyword Search:
- Matches exact words only
- Cannot understand sentence meaning
- Fails if wording changes
- Simpler but less intelligent

=====
CONCLUSION
=====

Embedding-based semantic search captures contextual meaning
better than traditional keyword search.

Even if users phrase questions differently, semantic embeddings
can still retrieve the most relevant answer using similarity scores.

This is one of the core technologies used in:
- AI Chatbots
- Retrieval-Augmented Generation (RAG)
- Vector Databases
- AI Search Engines
- Modern LLM Applications
```

#### Screenshot 4 – completed successfully

```
=====
CONCLUSION
=====

Embedding-based semantic search captures contextual meaning
better than traditional keyword search.

Even if users phrase questions differently, semantic embeddings
can still retrieve the most relevant answer using similarity scores.

This is one of the core technologies used in:
- AI Chatbots
- Retrieval-Augmented Generation (RAG)
- Vector Databases
- AI Search Engines
- Modern LLM Applications

=====
PROGRAM COMPLETED SUCCESSFULLY
=====
```

# FINAL CONCLUSION

This project suite demonstrates the internal working principles of modern Large Language Models.

## Core Concepts Learned:

- **Attention Mechanism**
- **Tokenization**
- **Sampling Strategies**
- **Prompt Engineering**
- **Hallucination in AI**
- **Semantic Search using Embeddings**
- **Cosine Similarity**
- **Transformer Architecture Basics**

These are the foundational building blocks of modern AI systems such as:

**GPT, Gemini, Claude, and LLaMA**

---

**End of Report**